

Inteligência Artificial

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br





[1]Faculdade de Sistemas de Informação - UFPA/CUNTINS

13 de outubro de 2026

Roteiro

Planejamento

Fundamentação
- Conceitos Fundamentais
- Contexto Histórico
- Estado da Arte

Agentes Inteligentes
- Conceitos Fundamentais
- Racionalidade
- Onisciência
- Natureza dos ambientes
- Propriedades de ambientes de tarefas
- Estrutura de Agente



Licença

Este trabalho está licenciado sob a licença Creative Commons:





Nesta aula

- O que é Inteligência Artificial, afinal? Vamos ver as diferentes formas de defini-la e por que essa definição ainda gera debate.
- Um panorama histórico: de onde a área veio e para onde está indo.
- O conceito central do curso: **agentes inteligentes**: como são definidos, o que os torna racionais e como se estruturam internamente.
- Exemplos de código ficam disponíveis em dois repositórios:
  - Do livro-texto: `https://github.com/aimacode`
  - Da disciplina (professor): `https://github.com/eltonsarmanho`



Conceitos Fundamentais



O que é Inteligência Artificial?

- Não existe uma definição única e consensual de IA, e isso não é um defeito, é um reflexo de que a própria noção de "inteligência" já é escorregadia mesmo quando falamos de seres humanos.
- A definição de Russell & Norvig, que seguimos neste curso, é operacional:
  - **IA é o estudo de agentes que recebem percepções do ambiente e executam ações** de modo a maximizar uma medida de desempenho.
- Essa definição desloca o foco de "o que é pensar" para "o que o sistema faz", e é por isso que o conceito de **agente** será o fio condutor de todo o curso.
- IA é, na prática, um guarda-chuva de subcampos: aprendizado de máquina, visão computacional, processamento de linguagem natural, robótica, planejamento, raciocínio sob incerteza, entre outros.



Quatro Formas de Encarar a IA

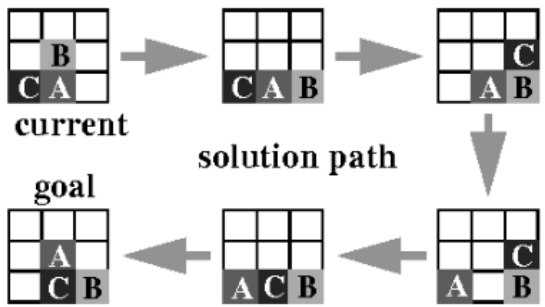
- Historicamente, as tentativas de definir IA se organizam em duas perguntas independentes:
  - O sistema deve **pensar** ou **agir**?
  - O padrão de comparação é o **ser humano** ou um ideal de **racionalidade**?
- Cruzando essas duas perguntas obtemos quatro escolas de pensamento, que vamos percorrer a seguir.

	Padrão: Humano	**Padrão: Racionalidade**
Pensar	Pensar Humanamente	Pensar Racionalmente
Agir	Agir Humanamente	Agir Racionalmente



Pensar Humanamente: a Abordagem Cognitiva

- A ideia aqui é literal: construir programas cujo *funcionamento interno* imite o raciocínio humano, não apenas o resultado final.
- Isso exige evidência empírica de como a mente funciona, obtida por introspecção, experimentos psicológicos e, mais recentemente, neuroimagem. É essa ambição que deu origem à **ciência cognitiva**.
- Exemplo clássico: o **GPS** (*General Problem Solver*, Newell e Simon, 1961) não só resolvia problemas, buscava reproduzir o *traço* de passos que humanos seguem ao resolvê-los.



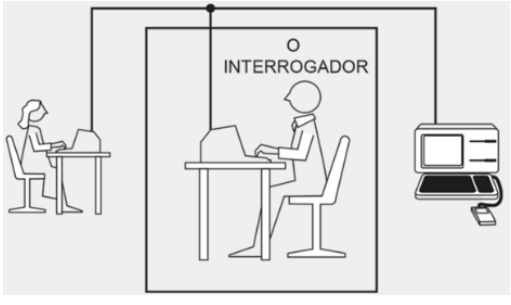
Pensar Racionalmente: as Leis do Pensamento

- Tradição que remonta a Aristóteles: especificar padrões de raciocínio inquestionavelmente corretos, os **silogismos**, e mecanizá-los.
- Deu origem à lógica formal do século XIX e, depois, aos primeiros programas capazes de provar teoremas a partir de premissas.
- **Duas dificuldades**: nem todo conhecimento cotidiano é fácil de formalizar (pense em incerteza), e, mesmo formalizado, resolver o problema pode ser computacionalmente inviável na prática.



Agir Humanamente: o Teste de Turing

- Alan Turing propôs, em 1950, contornar "máquinas podem pensar?" com um critério comportamental: o **Jogo da Imitação**.
- Um interrogador conversa, por texto, com um humano e com uma máquina sem saber qual é qual, e se não conseguir distingui-los, a máquina é aprovada.
- Isso exigiria PLN, representação de conhecimento, raciocínio automatizado e aprendizado de máquina.
- **Reflexão**: LLMs hoje conversam de forma fluente. Isso significa que "pensam"? O próprio Turing notava que o teste mede comportamento, não mecanismo interno.



Agir Racionalmente: o Agente Racional

- Um **agente racional** é aquele que age de modo a alcançar o melhor resultado esperado, dado o que percebeu e o que sabe.
- Essa noção é **mais geral** que as leis do pensamento: raciocinar corretamente é apenas *um dos meios* de agir racionalmente, pois às vezes a ação certa nem exige inferência lógica (um reflexo bem calibrado também pode ser racional).
- É também **mais tratável cientificamente** do que imitar humanos: dá para formalizar matematicamente o que é uma boa decisão e construir agentes que se aproximem dela, mesmo quando a solução ótima exata é computacionalmente inviável.
- Por essas razões, é a abordagem adotada neste curso, e por isso o restante da disciplina gira em torno do conceito de **agente**.



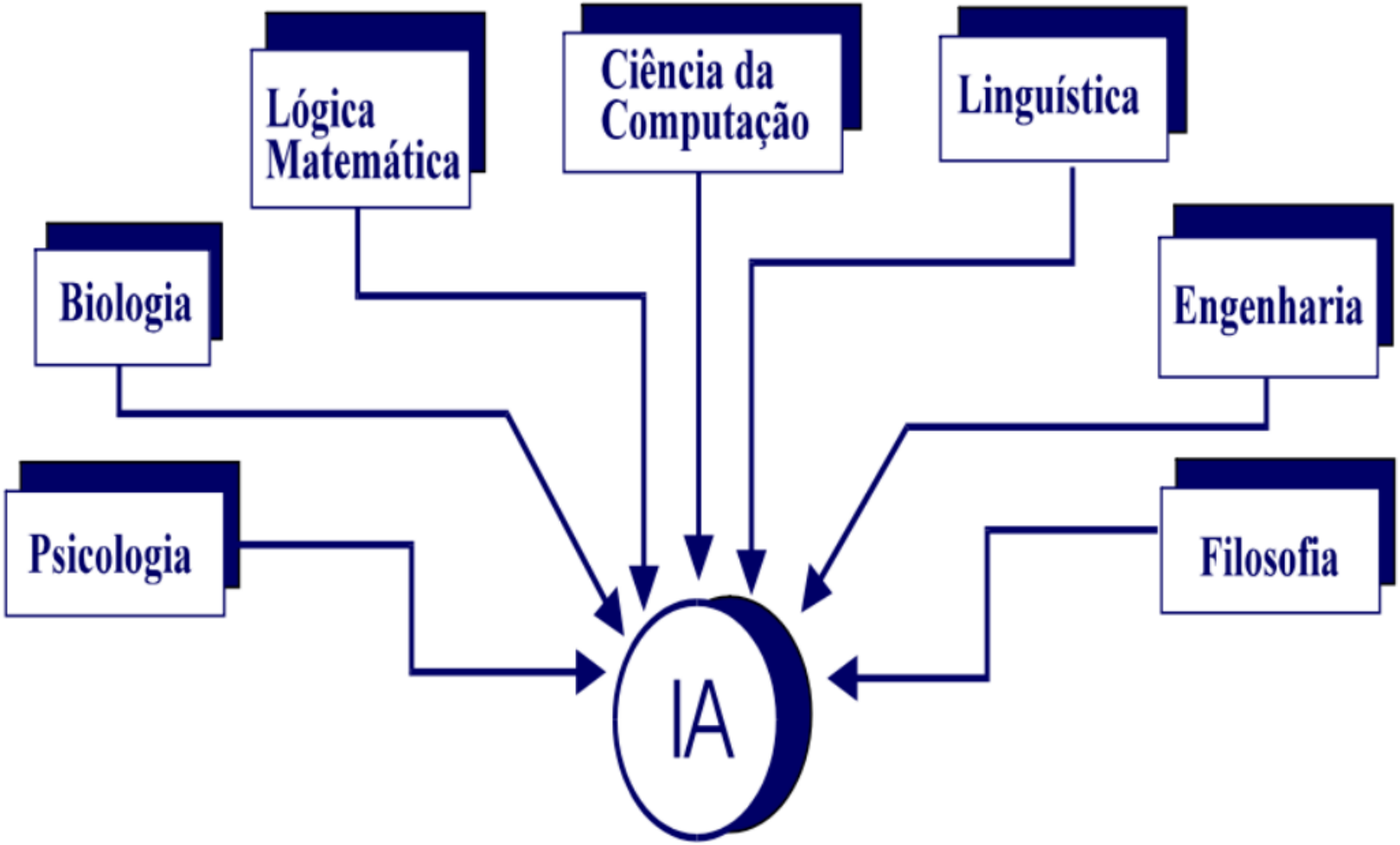
Resumo das Quatro Abordagens

Abordagem	**Ideia central**
Pensar Humanamente	Reproduzir o processo cognitivo humano (ciência cognitiva)
Pensar Racionalmente	Formalizar o raciocínio correto em lógica (leis do pensamento)
Agir Humanamente	Imitar o comportamento humano de forma indistinguível (Teste de Turing)
Agir Racionalmente	Escolher a melhor ação esperada, dado o conhecimento disponível

Este curso adota a abordagem de **agentes racionais**.



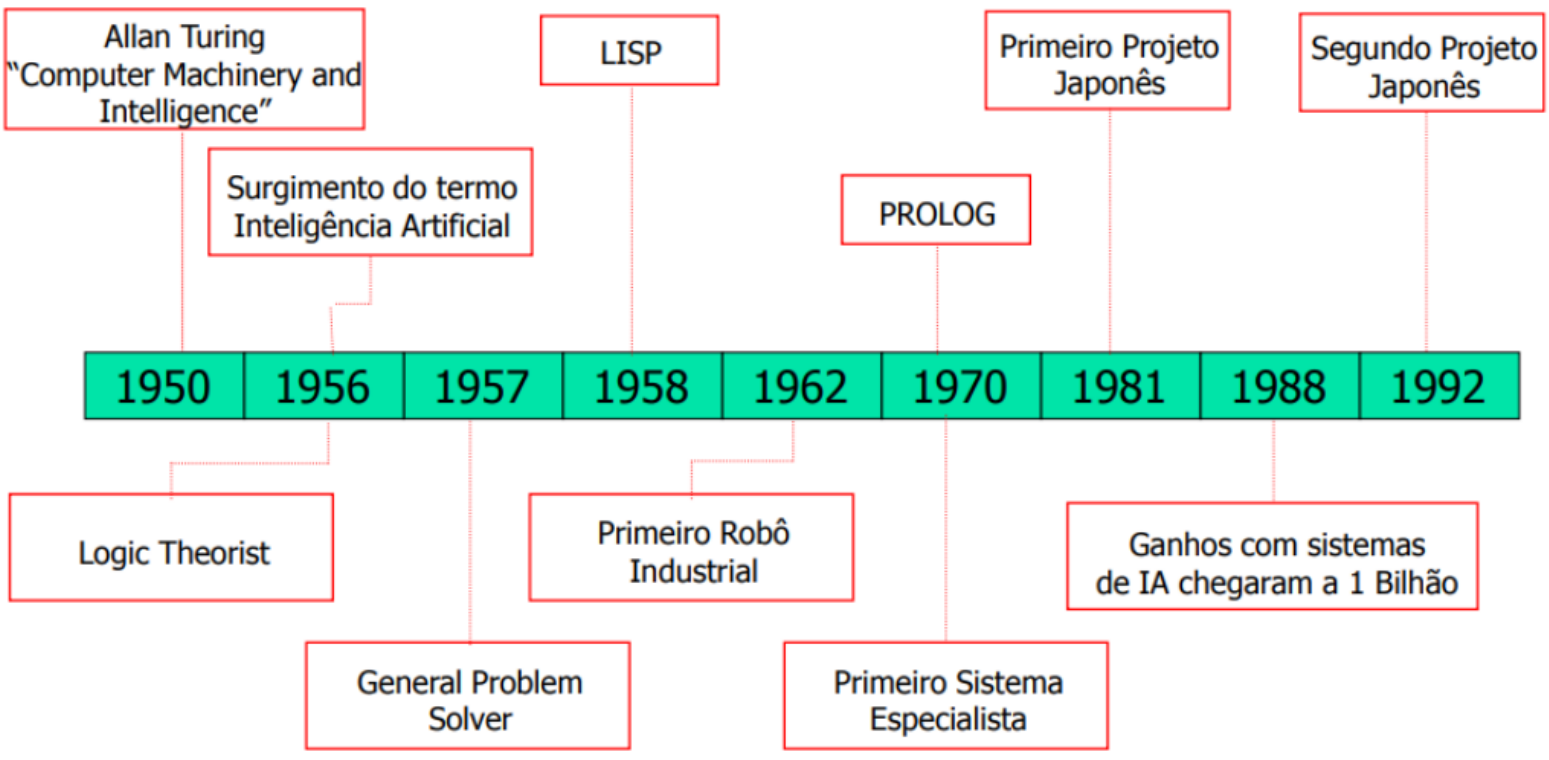
Áreas de Apoio para IA



Sub-Áreas da IA

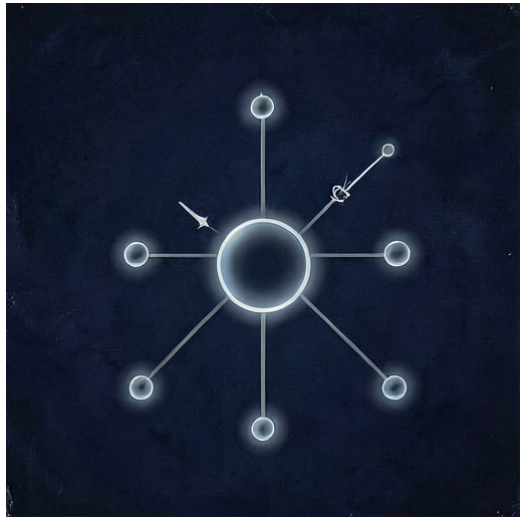


Linha do Tempo



Gestação da IA (1943–1955)

- McCulloch e Pitts (1943) propõem o primeiro modelo matemático de neurônio artificial.
- Hebb (1949) propõe a regra de aprendizagem que leva seu nome, ainda usada hoje.
- Minsky e Edmonds constroem o SNARC (1951), o primeiro computador neural.



Nascimento da IA (1956)

- McCarthy organiza o workshop de Dartmouth, o marco fundador da área.
- É ali que o termo **"Inteligência Artificial"** é cunhado.
- Participam nomes como Minsky, Shannon e Newell.



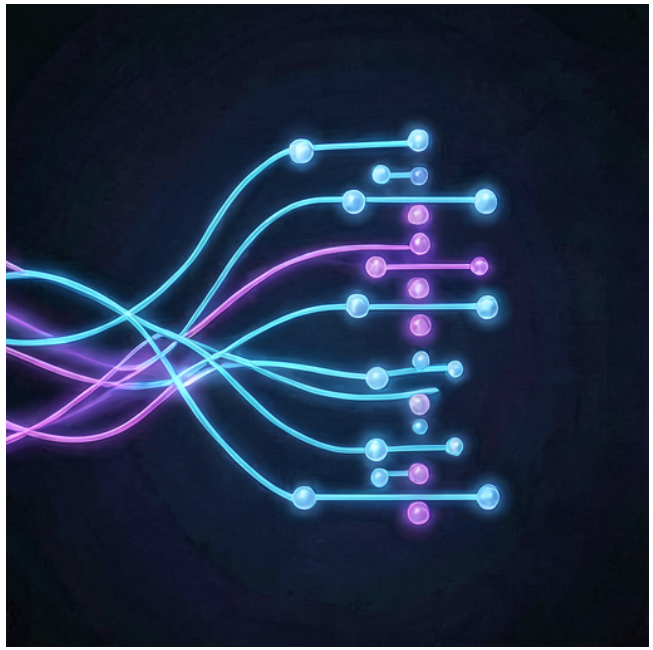
Entusiasmo Inicial (1952–1969)

- O **Logic Theorist** e o **GPS** provam teoremas e resolvem problemas como humanos.
- Arthur Samuel cria um programa de damas que aprende jogando contra si mesmo.



O Retorno das Redes Neurais (1986–presente)

- A **retropropagação** é redescoberta, permitindo treinar redes com várias camadas.
- Reacende o confronto entre IA simbólica e IA conexionista.



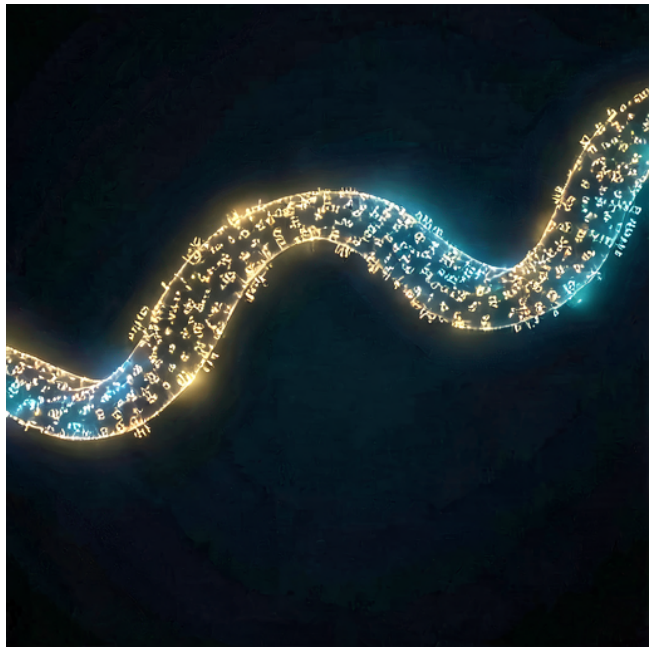
Método Científico e Big Data (1987–2012)

- A IA passa a exigir **rigor experimental**, aproximando-se de estatística e otimização.
- Com o **ImageNet** e GPUs, o **AlexNet** (2012) inicia a revolução do deep learning.



A Era dos Transformers e dos LLMs (2017–presente)

- "Attention Is All You Need" (2017) introduz o **Transformer**, base dos grandes modelos de linguagem.
- O ChatGPT (2022) populariza os **LLMs**, tornando a IA conversacional cotidiana.



O que está em alta agora

- A IA já teve muitos marcos históricos (Deep Blue, DART, Remote Agent). O foco aqui é outro: o que domina o noticiário e a pesquisa **hoje**.
- Quatro frentes concentram praticamente todo o hype atual: carros autônomos, LLMs, geração de conteúdo e agentes de IA.



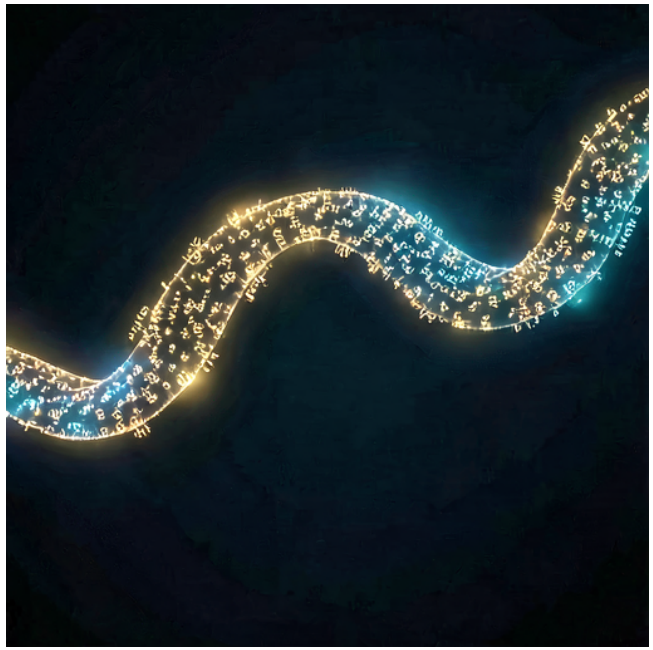
Condução Autônoma

- Waymo e Cruise já operam frotas de táxis **sem motorista** em várias cidades dos EUA.
- Tesla e outras montadoras oferecem condução assistida cada vez mais autônoma em carros comerciais.



Modelos de Linguagem (LLMs)

- **GPT**, **Gemini** e **Claude** conversam, programam, resumem e traduzem com alta fluência.
- Limitações a lembrar: **alucinações**, vieses herdados dos dados e alto custo computacional.



Geração de Imagem e Vídeo por IA

- Modelos de difusão (Midjourney, Sora e sucessores) geram imagens e vídeos realistas a partir de texto.
- Levanta questões novas: direitos autorais, *deepfakes* e verificação de autenticidade.



Agentes de IA: do Texto à Ação

- A fronteira atual: transformar um LLM em **agente**, que lê, decide e age (chama ferramentas, executa código, navega na web).
- LangChain e LangGraph orquestram esse ciclo, formando um **agente** no sentido de Russell & Norvig.



Conclusão

- O fio que conecta todo o hype atual é sempre o mesmo conceito: um sistema que **percebe** um ambiente e **age** sobre ele para atingir um objetivo.
- É exatamente esse conceito, o de **agente**, que vamos formalizar na próxima parte da aula.



Agentes Inteligentes



Uma Analogia antes da Formalização



Figura: Metáfora de um agente: um fluxo entra (percepção), atravessa a figura (decisão) e sai como ação, em ciclo contínuo com o ambiente.

- Antes de formalizar, vale fixar a intuição: um agente é qualquer coisa que **percebe** seu entorno e **age** sobre ele, de forma contínua, seja um robô, um termostato ou um programa.

Questão para reflexão

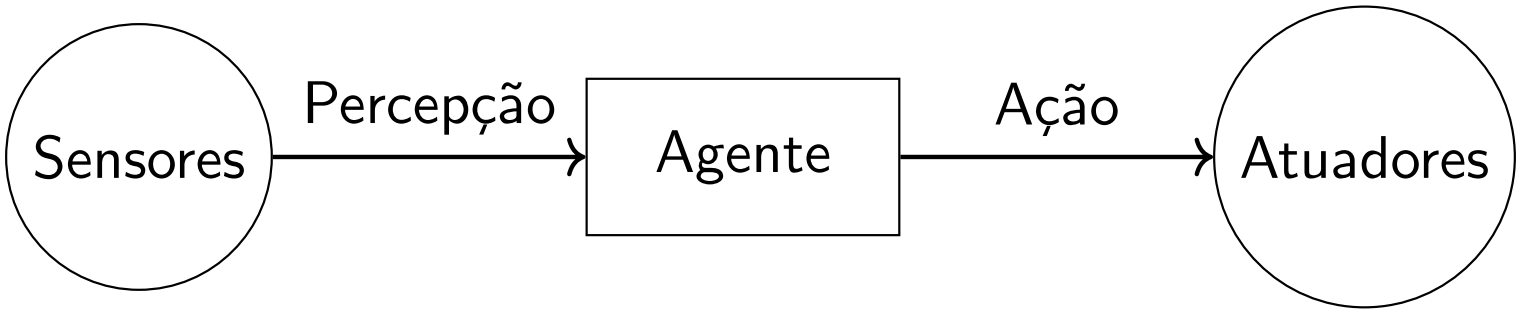
Pense em um app ou dispositivo do seu dia a dia. O que faz o papel de "sensor" e de "atuador" nele?

Definição de Agente

- **Agente:** Entidade que percebe seu ambiente por meio de sensores e age sobre ele por meio de atuadores.
- Representação básica:

$$\text{Percepção} \rightarrow \text{Processamento} \rightarrow \text{Ação}$$

$$\{s_1, s_2, \ldots, s_n\} \rightarrow f \rightarrow \{a_1, a_2, \ldots, a_m\}$$



Comportamento do Agente

- O **comportamento** de um agente é descrito por uma **função** que mapeia uma sequência de percepções em uma ação.
- **Função de Agente:** $f : P^* \rightarrow A$
  - Onde P^* é o conjunto de todas as sequências de percepções possíveis e A é o conjunto de ações.



Agente Inteligente

Um **agente inteligente** é um agente que:

- Toma decisões autônomas para atingir objetivos.
- Utiliza raciocínio e aprendizado para melhorar seu desempenho.

Racionalidade

Um agente é **racional** se ele escolhe a ação que maximiza sua **função de utilidade**:

$$\text{Ação ótima} = \arg\max_{a \in A} \sum_{s'} P(s' \mid s, a) \cdot U(s')$$

onde:

- s é o estado atual.
- s' é o estado futuro.
- $P(s' \mid s, a)$ é a probabilidade de transição.
- $U(s')$ é a utilidade do estado s'.



Autonomia

Um agente é **autônomo** se:

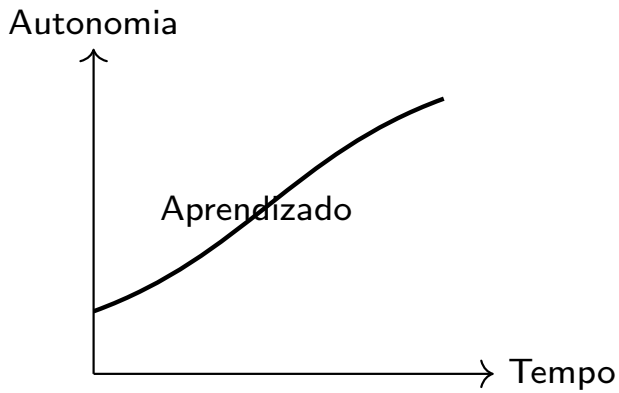
- Opera sem intervenção humana direta.
- Adapta-se a mudanças no ambiente.

Exemplo Matemático

Seja E o ambiente e K o conhecimento do agente. A autonomia pode ser modelada como:

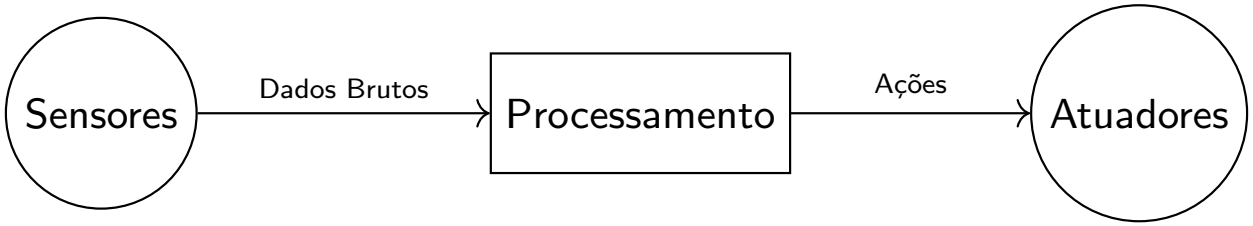
$$\text{Ação autônoma} = f(E, K)$$

onde f é a função de decisão do agente.



Percepção e Ação

- **Sensores** coletam dados do ambiente: câmeras, microfones, sensores de temperatura, acelerômetros.
- **Atuadores** permitem agir sobre o ambiente: motores, braços robóticos, alto-falantes.
- O **processamento de sinais** interpreta os dados brutos: filtragem de ruído, reconhecimento de padrões, extração de características.



Definição de Racionalidade

- Um agente é **racional** se escolhe, em cada instante, a ação que maximiza sua medida de desempenho esperada, dado o que percebeu até ali e o conhecimento que possui.
- Racionalidade não é onisciência: o agente decide com a informação que tem *no momento* da decisão, não com o que descobriria depois.
- Ela depende de três ingredientes que vamos detalhar a seguir: a **medida de desempenho**, o **conhecimento prévio** e a **sequência de percepções e ações possíveis**.



Exemplo: Agente Aspirador de Pó

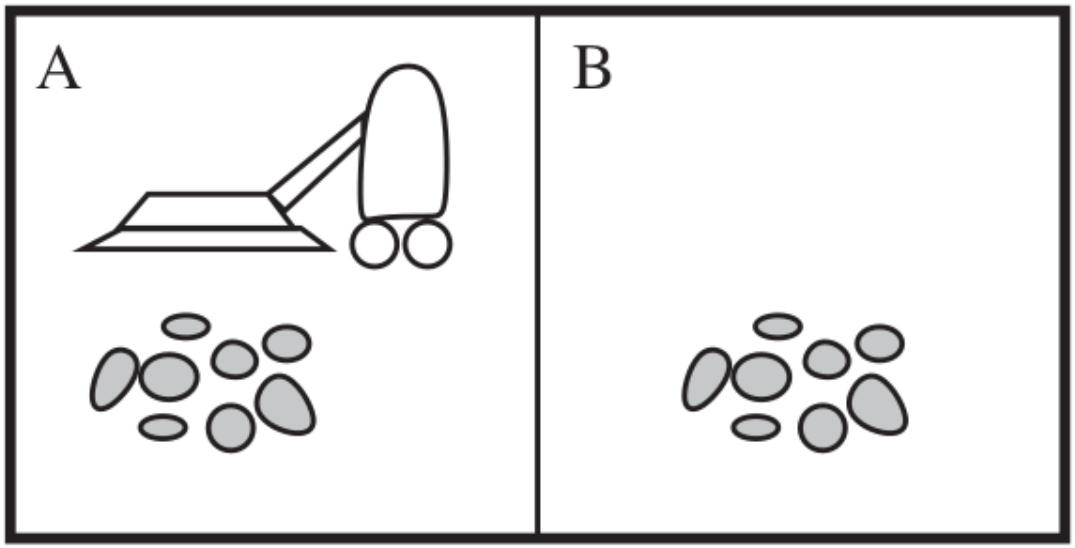


Figura: Mundo do aspirador: dois quadrados, cada um limpo ou sujo.

Exemplo: Agente Aspirador de Pó

- O agente aspirador limpa o quadrado onde está se ele estiver sujo, e se move para o outro quadrado caso contrário.
- A função do agente pode ser inteiramente descrita por uma tabela que mapeia cada percepção possível (local + estado) na ação correspondente.

Sequência de percepções	Ação
[*A*, *Limpo*]	*Aspirar*
[*A*, *Sujo*]	*Direita*
[*B*, *Limpo*]	*Aspirar*
[*B*, *Sujo*]	*Esquerda*
[*A*, *Limpo*], [*A*, *Limpo*]	*Aspirar*
[*A*, *Limpo*], [*A*, *Sujo*]	*Direita*
.	*Aspirar*
.	
.	
[*A*, *Limpo*], [*A*, *Limpo*], [*A*, *Limpo*]	*Direita*
[*A*, *Limpo*], [*A*, *Limpo*], [*A*, *Sujo*]	*Aspirar*
.	

Figura: Tabela parcial da função de um agente simples no mundo do aspirador de pó.



Medida de Desempenho

- É o critério objetivo usado para avaliar *o quão bem* o agente se saiu. Não existe uma medida universal: ela precisa ser definida para cada ambiente e objetivo.
- Exemplo: no mundo do aspirador, poderia ser a quantidade de sujeira removida, o tempo gasto ou a energia consumida ao longo de 1000 passos.

Princípio de projeto

De modo geral, é melhor definir a medida de desempenho em função do **resultado desejado no ambiente** *("casa limpa") do que em função do* **comportamento esperado do agente** *("aspirar sempre que possível"), pois a segunda abordagem pode recompensar um agente que age bem sem de fato atingir o objetivo.*



Racionalidade do Agente: Quatro Fatores

- A racionalidade de uma ação, em um dado instante, depende de quatro fatores:
  - A **medida de desempenho** que define o critério de sucesso.
  - O **conhecimento prévio** do agente sobre o ambiente.
  - As **ações** que o agente pode realizar.
  - A **sequência de percepções** que o agente teve até o momento.

Definição de agente racional

Para cada sequência de percepções possível, um agente racional deve escolher uma ação que maximize sua medida de desempenho esperada, dadas as evidências fornecidas pela sequência de percepções e todo o conhecimento prévio do agente.



Racionalidade e Otimalidade

- **Racionalidade** maximiza o desempenho *esperado*, com base no que o agente sabe no momento da decisão.
- **Otimalidade** exigiria maximizar o desempenho *real*, considerando toda informação relevante, inclusive a que o agente não tem como possuir.
- Por isso racionalidade e otimalidade nem sempre coincidem: limitações de tempo, conhecimento e capacidade computacional impedem que um agente seja perfeito, mas não impedem que ele seja racional.



Conhecimento Prévio

- O que o agente sabe sobre o ambiente antes de começar a operar.
- Inclui modelos do ambiente, regras de interação, e experiências passadas.
- Exemplo: Um agente de navegação pode conhecer a disposição das ruas, mas não as condições de tráfego em tempo real.



Ações Possíveis

- As ações que o agente pode realizar em resposta às percepções.
- A racionalidade de um agente inclui considerar todas as ações possíveis e escolher a melhor delas.
- Exemplo: Um robô móvel pode escolher entre avançar, virar ou parar com base nas percepções de seu sensor.



Sequência de Percepções

- A sequência de percepções passadas pode influenciar as decisões futuras do agente.
- Um agente racional utiliza essa sequência para atualizar seu conhecimento e refinar suas decisões.
- Exemplo: Um agente de investimento pode ajustar sua estratégia com base em padrões observados no mercado.



Racionalidade não é Onisciência

- Um agente **racional** escolhe a melhor ação com base no que percebeu e sabe, e não no que *acabaria* acontecendo.
- Um agente **onisciente** conheceria o resultado real de cada ação de antemão. Isso simplesmente não existe no mundo real.
- Exemplo clássico: alguém atravessa a rua numa faixa segura e, por puro azar, é atingido por uma peça que cai de um avião. A decisão de atravessar continua racional, pois não havia como prever aquele evento com a informação disponível.



Perfeição vs. Racionalidade

- Racionalidade maximiza o desempenho *esperado*, enquanto a perfeição exigiria maximizar o desempenho *real*, incluindo eventos impossíveis de antecipar.
- Exigir perfeição tornaria o projeto de agentes práticos inviável, já que nenhum sistema real consegue prever todos os desdobramentos futuros de suas ações.



Aprendizado e Autonomia

- **Aprendizado**: um agente racional deve conseguir melhorar seu desempenho ao longo do tempo, ajustando-se com base no que observa.
- **Autonomia**: um agente deve agir cada vez mais a partir da própria experiência, e não apenas do conhecimento que lhe foi programado de antemão.
- Exemplo: um robô de limpeza que nunca viu o ambiente consegue, explorando-o, descobrir sozinho onde a sujeira se acumula com mais frequência, e adaptar sua rota de acordo.



Definição do Ambiente de Tarefa

- O ambiente de tarefa de um agente inclui todos os aspectos externos ao agente que são relevantes para a definição do problema que ele deve resolver.
  - Ao projetar um agente, a primeira etapa deve ser sempre especificar o ambiente de tarefa chamado de **PEAS**
- **PEAS**:
  - P: Performance Measure (Medida de Desempenho)
  - E: Environment (Ambiente)
  - A: Actuators (Atuadores)
  - S: Sensors (Sensores)

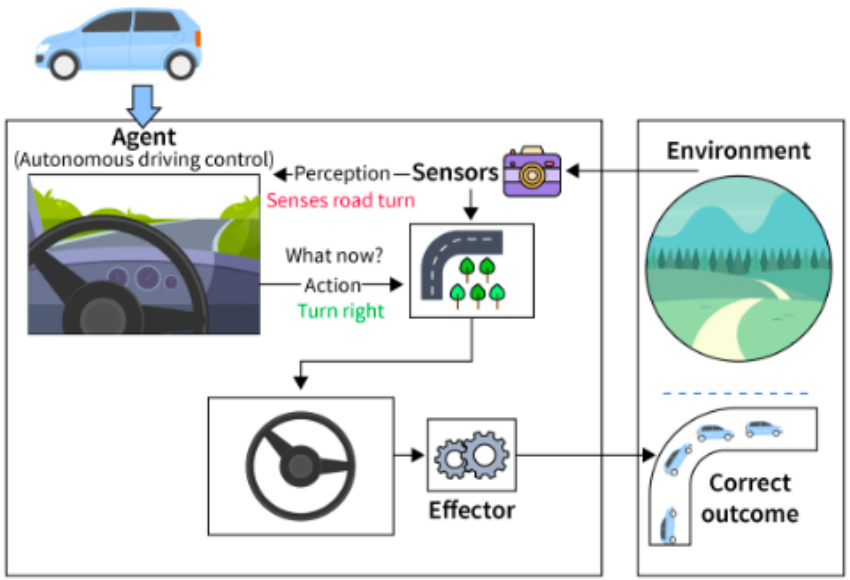


Exemplo de PEAS: Agente Aspirador de Pó

- Medida de Desempenho: Quantidade de sujeira removida, tempo gasto, energia consumida.
- Ambiente: Ambiente doméstico com áreas limpas e sujas.
- Atuadores: Motores de movimentação e sistema de sucção.
- Sensores: Sensores de sujeira, sensores de parede, sensores de localização.



PEAS: Agente de Condução Autônoma (ex: Uber)



P	Segurança, conformidade com as regras de trânsito, eficiência
E	Ruas, outros veículos, pedestres, sinais de trânsito
A	Motores, volante, freios
S	Câmeras, radar, GPS, sensores de velocidade

PEAS para Diferentes Ambientes

- Agente de Diagnóstico Médico:
  - P: Precisão do diagnóstico, tempo de resposta.
  - E: Histórico do paciente, sintomas apresentados, resultados de exames.
  - A: Interface de comunicação, recomendações de tratamento.
  - S: Entrada de dados de sintomas, resultados laboratoriais.



Tabela de Exemplos de PEAS

Agente	**P**	**E**	**A**	**S**
Aspirador de Pó	Limpeza	Casa	Motores, sucção	Sensores de sujeira
Condução Autônoma	Segurança	Ruas	Volante, freios	Câmeras, radar
Diagnóstico Médico	Precisão	Clínicas	Recomendações	Entrada de dados
Robô de Entrega de Pacotes	*?*	*?*	*?*	*?*

Tabela: Exemplos de PEAS, cuja última linha é o exercício a seguir.



Especificando o Ambiente de Tarefa

- A especificação clara do ambiente de tarefa é crucial para o design e a implementação de agentes racionais.
- Compreender os componentes do PEAS ajuda a definir os requisitos e as restrições de cada agente.
- Impacta diretamente na escolha de algoritmos e na arquitetura do agente.



Tarefa 1: Robô de Entrega de Pacotes na UFPA de Cametá

Imagine que você está desenvolvendo um robô autônomo capaz de entregar pacotes em um campus universitário. O robô deve navegar pelos prédios, evitar obstáculos, e garantir que os pacotes sejam entregues no local certo.



Questão 1: Medida de Desempenho (Performance Measure)

- Como você definiria a medida de desempenho para este robô de entrega?
- Quais métricas devem ser usadas para avaliar se o robô está executando bem sua tarefa?



Questão 2: Ambiente (Environment)

- Descreva o ambiente no qual o robô operará.
- Quais são os elementos do ambiente que o robô precisará considerar ao tomar decisões?



Questão 3: Atuadores (Actuators)

- Quais atuadores o robô precisará para realizar suas ações?
- Considere todos os tipos de movimentos e interações que o robô terá com o ambiente.



Questão 4: Sensores (Sensors)

- Quais sensores o robô precisará para perceber o ambiente e tomar decisões informadas?
- Considere tanto sensores para navegação quanto para interação com os objetos e pessoas.



Solução: PEAS do Robô de Entrega de Pacotes

P	Pacotes entregues no prazo e no local certo, sem danos, tempo total gasto e energia consumida.
E	Prédios, corredores, rampas, elevadores, pessoas e outros obstáculos do campus.
A	Motores de locomoção, braço ou compartimento para soltar o pacote, display/alto-falante para interagir com pessoas.
S	Câmeras, LiDAR ou sonares para navegação, GPS/mapa interno, leitor de código do pacote, sensores de contato/colisão.

Tabela: Uma resposta possível: compare com a sua.



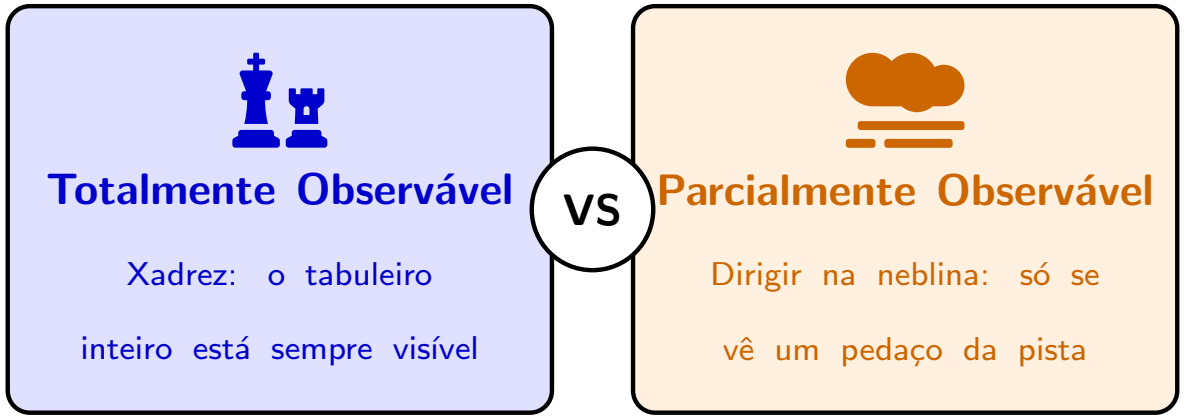
Propriedades dos Ambientes de Tarefa

- Depois de especificar o PEAS, o próximo passo é caracterizar o **ambiente** segundo um conjunto de dimensões, pois isso é o que realmente determina qual algoritmo e qual arquitetura de agente fazem sentido.
- As seis dimensões a seguir formam um "checklist" que vale a pena aplicar a qualquer ambiente novo que vocês encontrarem.



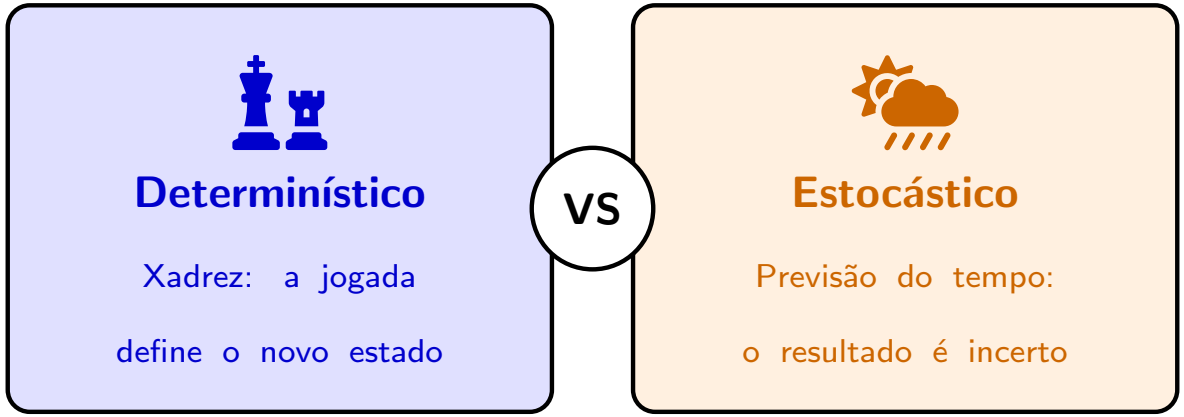
Totalmente Observável vs. Parcialmente Observável

- **Totalmente observável**: os sensores do agente dão acesso ao estado completo do ambiente, relevante para a decisão, a cada instante.
- **Parcialmente observável**: a informação é incompleta, ruidosa ou simplesmente inacessível (sensores limitados, ambiente oculto).



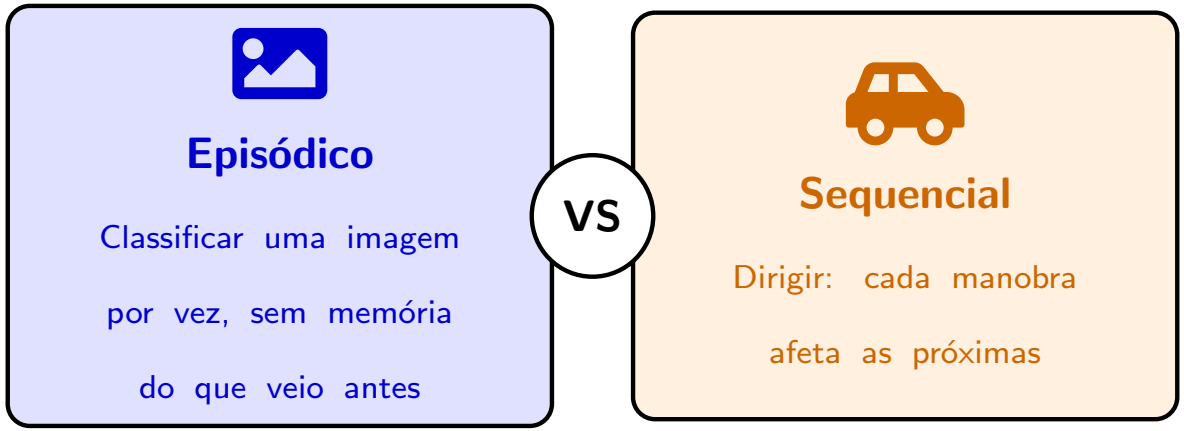
Determinístico vs. Estocástico

- **Determinístico**: o próximo estado é inteiramente definido pelo estado atual e pela ação do agente, sem acaso envolvido.
- **Estocástico**: há incerteza genuína no resultado da ação, de modo que o próximo estado segue uma distribuição de **probabilidades**.



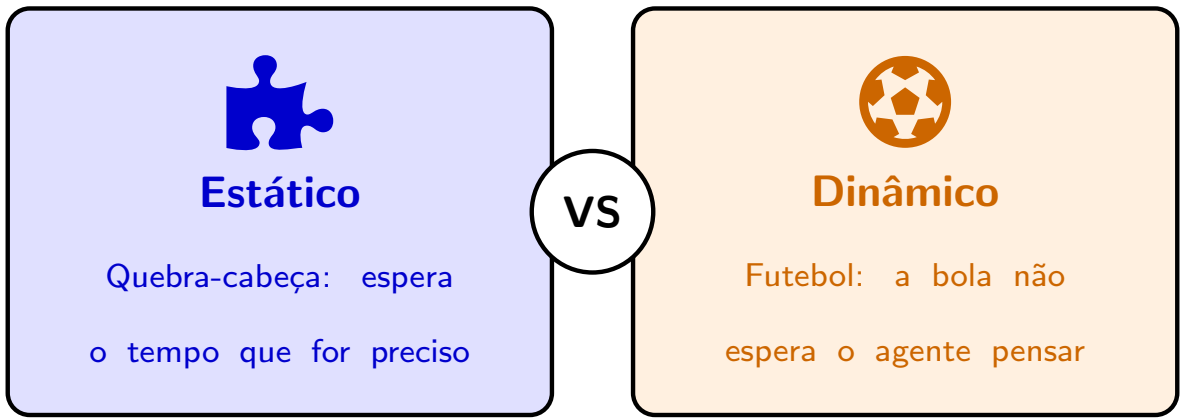
Episódico vs. Sequencial

- **Episódico**: a experiência do agente é dividida em episódios independentes, de modo que a decisão de um episódio não afeta os seguintes.
- **Sequencial**: a decisão atual pode afetar todas as decisões futuras, exigindo pensar à frente.



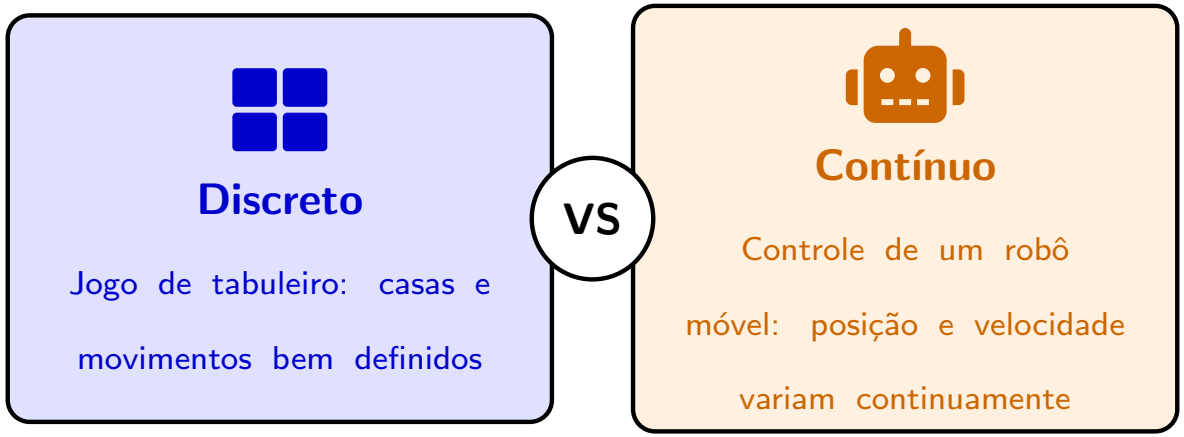
Estático vs. Dinâmico

- **Estático**: o ambiente permanece parado enquanto o agente delibera, de modo que não há pressa em decidir.
- **Dinâmico**: o ambiente pode mudar durante a deliberação do agente, que precisa decidir em tempo hábil.



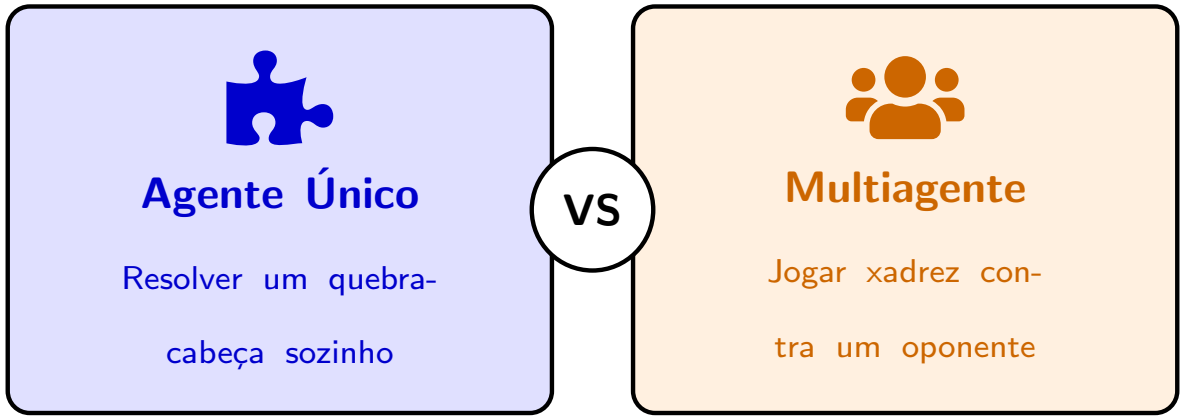
Discreto vs. Contínuo

- **Discreto**: número finito (e geralmente pequeno) de estados, percepções e ações distintas.
- **Contínuo**: estados, tempo ou ações variam em um espaço contínuo, potencialmente infinito.



Agente Único vs. Multiagente

- **Agente único**: o agente opera sozinho, e o ambiente não contém outros agentes tomando decisões.
- **Multiagente**: outros agentes atuam no mesmo ambiente, de forma cooperativa (um time) ou competitiva (um adversário).



Outras Propriedades

- **Conhecido vs. Desconhecido**: um ambiente é conhecido quando o agente já sabe, de antemão, como suas ações afetam o ambiente (as "regras do jogo"), e é desconhecido quando o agente precisa explorar para aprender essas regras.
- Essa dimensão é diferente de observabilidade: é possível ter um ambiente totalmente observável e, ainda assim, desconhecido (por exemplo, um jogo novo cujas regras exatas não foram explicadas).
- Exemplo: Jogo de xadrez para um jogador experiente (conhecido) vs. explorar a superfície de Marte pela primeira vez (desconhecido).



Impacto no Design de Agentes

- Cada dimensão empurra o projeto do agente em uma direção: ambientes parcialmente observáveis exigem manter algum estado interno, ambientes dinâmicos exigem decisões rápidas e ambientes estocásticos exigem raciocinar sobre probabilidades.
- **O caso mais difícil** combina tudo de uma vez: **parcialmente observável, multiagente, estocástico, sequencial, dinâmico, contínuo e desconhecido**.
  - Exemplo: dirigir em um país desconhecido, com geografia, leis de trânsito e comportamento dos outros motoristas que o agente ainda não domina.
- Boa parte da pesquisa em IA existe justamente para lidar com combinações difíceis como essa, e é o que veremos nas próximas aulas.



Arquitetura e Programa de Agentes

- O trabalho da IA é projetar um **programa de agente**: o algoritmo que implementa a função do agente, mapeando percepções em ações.
- Esse programa roda sobre uma **arquitetura**, o dispositivo físico com sensores e atuadores (um PC, um carro robótico, um celular).

$$\textbf{agente} = \textbf{arquitetura} + \textbf{programa}$$

- O programa precisa ser compatível com a arquitetura: se ele recomenda a ação "andar", a arquitetura precisa ter pernas.



O que é um Programa de Agente

- É o algoritmo que recebe a percepção atual como entrada e devolve uma ação, a cada ciclo de execução do agente.
- Note a diferença: a **função** do agente é uma descrição matemática abstrata ($f : P^* \rightarrow A$), enquanto o **programa** é a implementação concreta que roda na arquitetura.



Pseudocódigo: Agente Dirigido por Tabela

- A implementação mais direta de um programa de agente é uma **tabela**: para cada sequência de percepções já vista, uma ação associada.

```
function TABLE-DRIVEN-AGENT(percept) returns action
  persistent: percepts, uma lista, inicialmente vazia
              table, uma tabela de ações, indexada por
                     sequências de percepções

  append percept ao final de percepts
  action <- LOOKUP(percepts, table)
  return action
```

- **Limitação séria**: a tabela cresce exponencialmente com o número de percepções possíveis, tornando essa abordagem inviável na prática para a maioria dos ambientes.



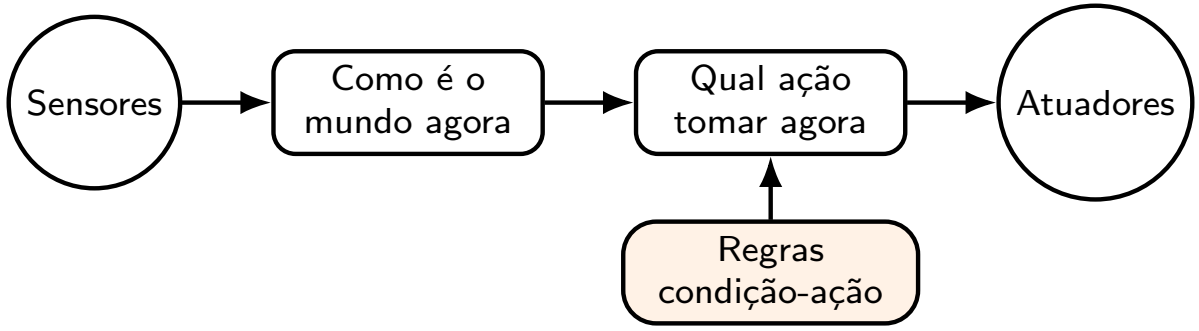
Tipos de Programas de Agentes

- Existem quatro famílias clássicas de programas de agentes, em ordem crescente de complexidade e de capacidade de lidar com ambientes difíceis:
  - **Reflexo simples**: decide com base só na percepção atual.
  - **Baseado em modelo**: mantém um estado interno sobre o que não pode observar diretamente.
  - **Baseado em objetivos**: decide prevendo se a ação aproxima do objetivo.
  - **Baseado em utilidade**: decide comparando o quão *bom* é cada resultado possível, não só se ele atinge o objetivo.



Agentes Simples de Reflexo

- Escolhem a ação com base apenas na percepção atual, por meio de **regras condição-ação** (“se sujo, aspire”).
- Funcionam bem apenas quando o ambiente é **totalmente observável**: a decisão correta precisa depender só do que está sendo percebido agora.
- Limitação: sem memória do passado, podem entrar em loops em ambientes parcialmente observáveis (ex: ficar alternando entre dois estados). Uma saída parcial é introduzir **aleatoriedade** nas escolhas.



Pseudocódigo: Agente de Reflexo Simples

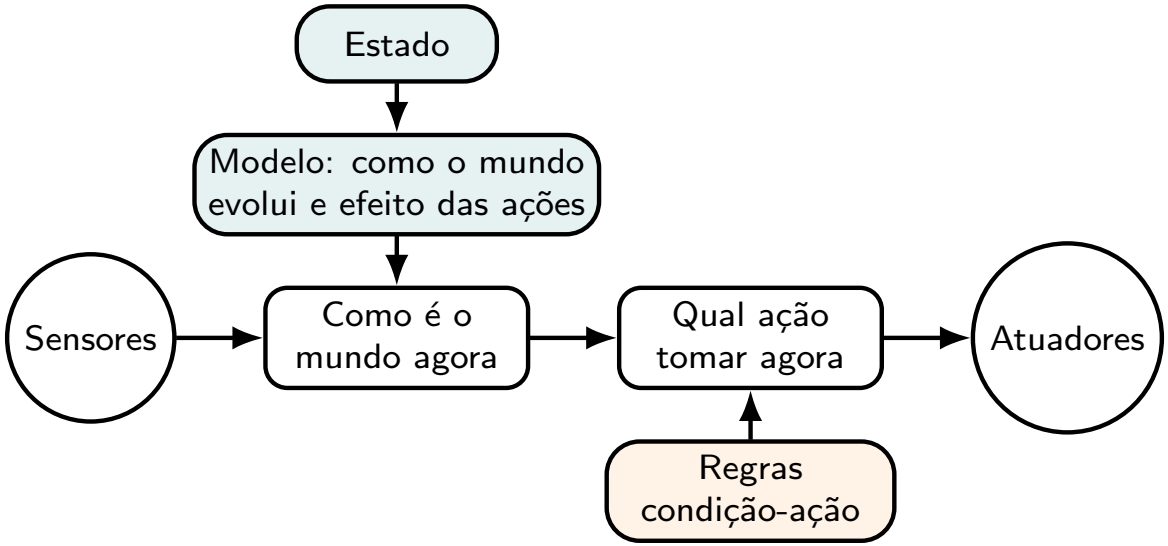
- Exemplo no mundo do aspirador: a ação depende só de onde o agente está e se o local está sujo.

```
function REFLEX-AGENT(location, status) returns action
  if status = SUJO then return ASPIRAR
  else if location = A then return DIREITA
  else if location = B then return ESQUERDA
```



Agentes Baseados em Modelo

- Mantêm um **estado interno** que guarda parte do histórico, combinado com um **modelo** de como o mundo evolui e do efeito das próprias ações.
- São mais robustos que os de reflexo simples: conseguem agir mesmo quando a percepção atual, sozinha, não basta para decidir.



Modelo do Ambiente

Um agente baseado em modelo mantém, internamente, uma função que descreve a transição de estados:

$$M : S \times A \rightarrow S$$

onde S é o conjunto de estados possíveis do ambiente e A é o conjunto de ações do agente.



Pseudocódigo: Agente de Reflexo com Modelo

```
function MODEL-BASED-REFLEX-AGENT(percept) returns action
  persistent: state,  a concepção atual do agente sobre o mundo
              model,  como o próximo estado depende de
                      estado e ação
              rules,  um conjunto de regras condição-ação
              action, a última ação, inicialmente nenhuma

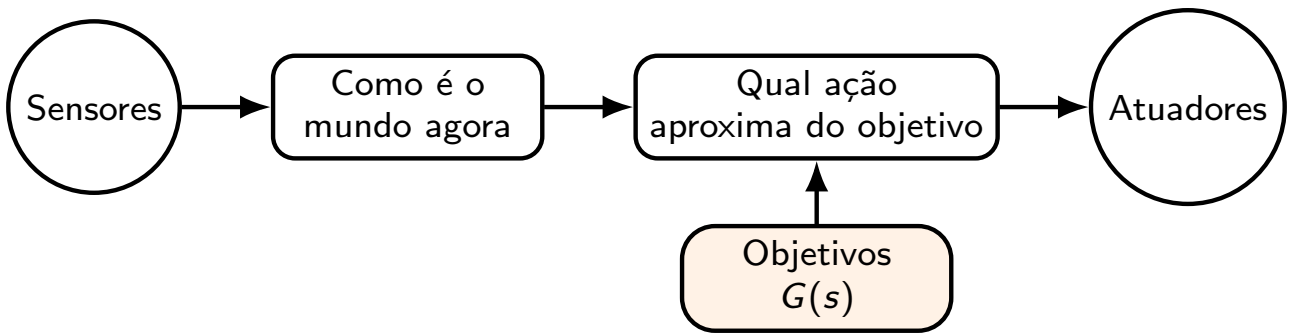
  state <- UPDATE-STATE(state, action, percept, model)
  rule  <- RULE-MATCH(state, rules)
  action <- rule.ACTION
  return action
```



Agentes Baseados em Objetivos

- Além do estado atual, consideram um **objetivo**: o estado que desejam alcançar. A ação escolhida é a que aproxima desse objetivo, não apenas a reação imediata.
- Exemplo: um robô de entrega decide a melhor **rota** até o destino, em vez de apenas reagir ao obstáculo mais próximo.
- Para isso, costumam usar **algoritmos de busca e planejamento** (assunto de aulas futuras).

$$G(s) = \begin{cases} 1 & \text{se } s \text{ é um estado objetivo} \\ 0 & \text{caso contrário} \end{cases}$$



Pseudocódigo: Busca em Largura (BFS)

- Exemplo clássico de algoritmo usado por agentes baseados em objetivos para encontrar um caminho até a meta:

```
function BFS(start, goal) returns a path
  frontier <- fila com [start]
  explored <- conjunto vazio

  while frontier não está vazia:
    node <- frontier.pop()
    if node = goal then return solution

    for each neighbor in node.neighbors:
      if neighbor not in explored:
        frontier.push(neighbor)
        explored.add(neighbor)
```



Vantagens e Limitações

- **Vantagens**: mais flexíveis que o reflexo simples, planejam à frente e consideram várias alternativas antes de agir.
- **Limitações**: a busca pela solução ideal pode exigir muito poder computacional, prever resultados é difícil em ambientes estocásticos, e os planos precisam ser atualizados quando o ambiente muda.

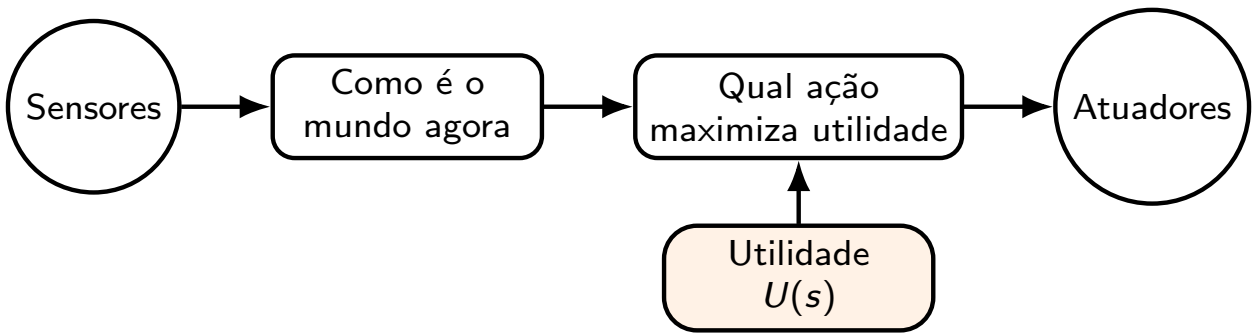


Agentes Baseados em Utilidade

- Vão além de um objetivo binário ("atingi ou não atingi"): avaliam o quão **bom** é cada estado possível, por meio de uma **função de utilidade**.
- Útil quando há vários objetivos conflitantes (rapidez vs. segurança) ou resultados incertos a comparar.

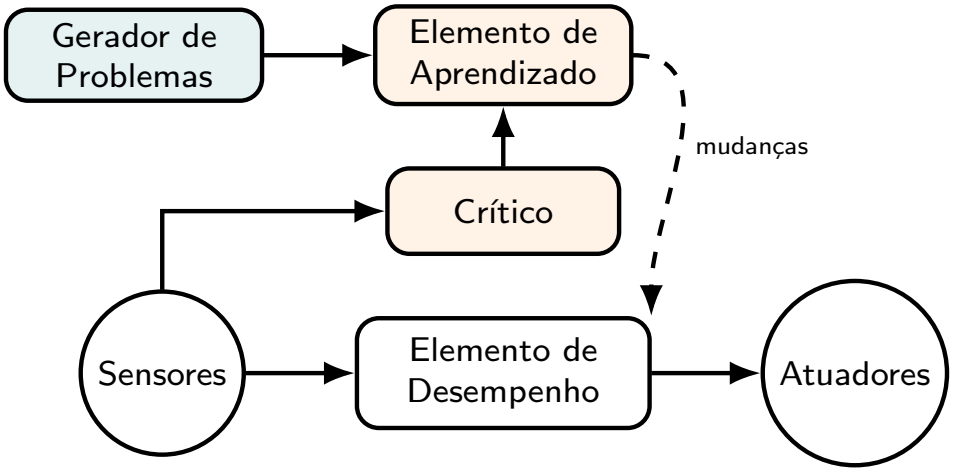
$$U(s) = \sum_i w_i \cdot f_i(s)$$

onde w_i são pesos e $f_i(s)$ são características do estado s.



Agentes de Aprendizado

- Qualquer um dos quatro tipos anteriores pode virar um **agente de aprendizado**, ganhando a capacidade de melhorar com a experiência em vez de depender só de conhecimento pré-programado.
- Quatro componentes trabalham juntos: o **elemento de desempenho** escolhe as ações, o **crítico** avalia o resultado, o **elemento de aprendizado** ajusta o comportamento futuro, e o **gerador de problemas** sugere ações exploratórias para descobrir coisas novas.



Bibliotecas Clássicas para Agentes em Python

- **AIMA-Python**: implementações de referência dos algoritmos e agentes do livro-texto do curso.
  - `https://github.com/aimacode/aima-python`
- **Mesa**: framework para simulação de agentes em ambientes 2D (modelagem baseada em agentes).
  - `https://mesa.readthedocs.io/en/stable/`
- **SPADE**: plataforma para agentes que se comunicam entre si pelo padrão FIPA, útil para sistemas multiagente.
  - `https://github.com/javipalanca/spade`
- **Gymnasium** (sucessor mantido do antigo OpenAI Gym): ambientes padronizados para treinar agentes com aprendizado por reforço.
  - `https://gymnasium.farama.org/`



Da Teoria aos Agentes Baseados em LLM

- As bibliotecas acima implementam a teoria clássica de agentes (AIMA). Hoje, uma nova geração de frameworks faz o mesmo papel usando um LLM como "cérebro" do agente.
- **LangChain** e **LangGraph**: orquestram o ciclo percepção-decisão-ação em torno de um LLM, dando a ele acesso a ferramentas, memória e fluxos de decisão com estado.
  - `https://python.langchain.com/`
- **Agno**: framework mais recente e leve, focado em agentes multiagente com memória, base de conhecimento e ferramentas prontas, com foco em velocidade de execução.
  - `https://www.agno.com/`
- Essas ferramentas serão exploradas em profundidade em aulas futuras desta disciplina, com exemplos práticos de agentes de IA em Python.



Referências I

- Chollet, F. (2021). Deep Learning with Python, Second Edition. Shelter Island, NY: Manning Publications.
- Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. Sebastopol, CA: O'Reilly Media.
- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4ª ed.). Hoboken, NJ: Pearson.
- Moroney, L. (2020). AI and Machine Learning for Coders. Sebastopol, CA: O'Reilly Media.



Referências II


Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).





Inteligência Artificial

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br





[1]Faculdade de Sistemas de Informação - UFPA/CUNTINS

13 de outubro de 2026